

Analyzing Raw Logs with Splunk

Author: Michael Miller

Current Date: April 29th, 2026

Purpose: To utilize Splunk as a method to ingest raw firewall logs and analyze for malicious activity.

Tools Used: Splunk

Firewall logs are simulated and were taken from https://github.com/splunk/attack_data. Credit goes to RavenTait for producing these logs.

Overview

I analyzed a raw firewall log, using Splunk's native indexing to break up the text file into manageable sections for proper analysis. The goal was to investigate any malicious activity in the network.

About the Data Source

This log was downloaded in plain text, and required proper formatting to analyze.

```
2014-11-08 23:52:53\tLocal2.Info\t10.139.6.24\t11Nov 8 23:52:52 1,2014/11/08 23:52:52,0008C101809,TRAFFIC,drop,1,2014/11/08 23:52:51
23:52:52,0,any,0,195151280,0x0,10.0.0.0(unresolved)-10.255.255.255(unresolved),10.0.0.0(unresolved)-10.255.255.255(unresolved),0,1,
2014-11-08 23:52:53\tLocal2.Info\t10.139.6.24\t11Nov 8 23:52:52 1,2014/11/08 23:52:52,0008C101809,TRAFFIC,drop,1,2014/11/08 23:52:51
23:52:52,0,any,0,195151280,0x0,10.0.0.0(unresolved)-10.255.255.255(unresolved),10.0.0.0(unresolved)-10.255.255.255(unresolved),0,1,
2014-11-08 23:52:53\tLocal2.Info\t10.139.6.24\t11Nov 8 23:52:53 1,2014/11/08 23:52:53,0008C101809,TRAFFIC,end,1,2014/11/08 23:52:52,
States,10.0.0.0(unresolved)-10.255.255.255(unresolved),0,1,1<000>\r\n
2014-11-08 23:52:53\tLocal2.Info\t10.139.6.24\t11Nov 8 23:52:53 1,2014/11/08 23:52:53,0008C101809,TRAFFIC,end,1,2014/11/08 23:52:52,
States,10.0.0.0(unresolved)-10.255.255.255(unresolved),0,1,1<000>\r\n
2014-11-08 23:52:53\tLocal2.Info\t10.139.6.24\t11Nov 8 23:52:53 1,2014/11/08 23:52:53,0008C101809,TRAFFIC,end,1,2014/11/08 23:52:53,
23:52:17,33,any,0,195151295,0x0,10.0.0.0(unresolved)-10.255.255.255(unresolved),10.0.0.0(unresolved)-10.255.255.255(unresolved),0,3
2014-11-08 23:52:53\tLocal2.Info\t10.139.6.24\t11Nov 8 23:52:53 1,2014/11/08 23:52:53,0008C101809,TRAFFIC,end,1,2014/11/08 23:52:53,
23:52:17,33,any,0,195151295,0x0,10.0.0.0(unresolved)-10.255.255.255(unresolved),10.0.0.0(unresolved)-10.255.255.255(unresolved),0,3
2014-11-08 23:52:53\tLocal2.Info\t10.139.6.24\t11Nov 8 23:52:53 1,2014/11/08 23:52:53,0008C101809,TRAFFIC,end,1,2014/11/08 23:52:53,
23:52:20,30,any,0,195151305,0x0,10.0.0.0(unresolved)-10.255.255.255(unresolved),10.0.0.0(unresolved)-10.255.255.255(unresolved),0,5
2014-11-08 23:52:53\tLocal2.Info\t10.139.6.24\t11Nov 8 23:52:53 1,2014/11/08 23:52:53,0008C101809,TRAFFIC,end,1,2014/11/08 23:52:53,
23:52:20,30,any,0,195151305,0x0,10.0.0.0(unresolved)-10.255.255.255(unresolved),10.0.0.0(unresolved)-10.255.255.255(unresolved),0,5
```

Because the logs were comma separated, I knew I could use indexing to create the categories I needed. I used Splunk to parse the comma-separated logs and leveraged AI to help construct the initial field extraction logic.

New Search

Save As ▾

Create Table View

Close

Time range: All time ▾

🔍

✖

```

index=attack.1
| eval f=split(_raw,",")
| eval log_time=mvindex(f,0)
| eval serial=mvindex(f,1)
| eval log_type=mvindex(f,2)
| eval subtype=mvindex(f,3)
| eval src_ip=mvindex(f,7)
| eval dest_ip=mvindex(f,8)
| eval src_zone=mvindex(f,16)
| eval dest_zone=mvindex(f,17)
| eval app=mvindex(f,14)
| eval protocol=mvindex(f,29)
| eval action=mvindex(f,30)
| eval bytes=mvindex(f,31)
| eval packets=mvindex(f,34)
| table _time log_type subtype src_ip dest_ip src_zone dest_zone app protocol action bytes packets

```

✓ 1,000 events (before 4/29/26 5:33:54.000 AM) No Event Sampling ▾

Job ▾ ||| ↻ ⚙️ 📄 Verbose Mode ▾

Events (1,000) Patterns **Statistics (1,000)** Visualization

Show: 20 Per Page ▾

Format ▾

Preview: On

◀ Prev

1

2

3

4

5

6

7

8

...

Next ▶

_time ▾	log_type ▾	subtype ▾	src_ip ▾	dest_ip ▾	src_zone ▾	dest_zone ▾	app ▾	protocol ▾	action ▾	bytes ▾	packets ▾
2025-11-08 23:53:12	0008C101802	TRAFFIC	10.138.16.29(SAL-UVS-REPOAP1)	10.138.32.32(unresolved)	UAT_LB_App	UAT_Inf_Servcs	incomplete	tcp	allow	70	2
2025-11-08 23:53:12	0008C101802	TRAFFIC	10.138.0.107(SAL-DVS-TCPRN04)	10.138.32.32(unresolved)	Dev_App	UAT_Inf_Servcs	incomplete	tcp	allow	70	2
2025-11-08 23:53:12	0008C101802	TRAFFIC	10.131.16.72(sal-vs-solar02.corp.svbank.com)	10.141.5.26(unresolved)	Inf_Services	Prod_In	snmp-base	udp	allow	188	3
2025-11-08 23:53:12	0008C101802	TRAFFIC	10.131.16.72(sal-vs-solar02.corp.svbank.com)	10.141.5.26(unresolved)	Inf-MGT_In	Inf-MGT_Ilo	snmp-base	udp	allow	188	3
2025-11-08 23:53:12	0008C101802	TRAFFIC	10.131.16.72(sal-vs-solar02.corp.svbank.com)	10.142.20.44(unresolved)	Inf_Services	Old_Prod_Temp	snmp-base	udp	allow	312	3
2025-11-08 23:53:12	0008C101802	TRAFFIC	10.131.16.72(sal-vs-solar02.corp.svbank.com)	10.131.40.175(unresolved)	Inf_Services	Prod_In	snmp-base	udp	allow	312	3

This quantifiably cut down on the time it took me to perform this analysis, and allowed me a bird's eye view of the data. The following fields were displayed:

1. Time
2. Log type
3. Subtype (traffic type)
4. Source IP
5. Destination IP
6. Source Zone
7. Destination Zone
8. App
9. Protocol
10. Action
11. Bytes
12. Packets

Item 6, 7, and 10 provide strong evidence that this is in fact a firewall log.

Methodology and Observation

Filtering Large Events

I began searching for large network connections. Splunk's heat map function identified one on the first page, with **108,417 bytes** exchanged in the session. The session below indicates TLS traffic from an internal IP at the bank to an external IP over SSL/TCP, permitted by the firewall. Nothing inherently suspicious.

2025-11-08 23:53:12	0008C101802	TRAFFIC	10.131.20.41(sal-vs-xappwk11.corp.svbank.com)	216.194.120.143(216-194-120-143.c7dc.com)		
Low_App	Prod_In	ssl	tcp	allow	108417	164

Because I only had access to the firewall log in this scenario, I next filtered for the highest bytes exchanged. There were two transfers of note, with **1,231,133 bytes** being exchanged. In the network exchange below this, there was an exchange for **210,124 bytes**, making the former the largest exchange on the logs by a wide margin, and an outlier. This does not prove malicious activity, but by combining the size and direction (egress), it is suspicious.

2025-11-08 23:52:57	0008C101809	TRAFFIC	139.131.195.10(unresolved)	208.80.234.202(unresolved)		
2025-11-08 23:52:57	0008C101809	TRAFFIC	139.131.195.10(unresolved)	208.80.234.202(unresolved)		
x_eCommerce_Out	x_eConnect_LB	ssl	tcp	allow	1231133	2782
x_eCommerce_Out	x_eConnect_LB	ssl	tcp	allow	1231133	2782

An initial search for any traffic that contained the 208 address as the destination IP or the 139 address as the source IP produced no results, nor did broadening it to include these IPs. Because this is a simulated attack, no tangible results were produced from a web search of the external IP 208.80.234.202.

Identifying Firewall Alerts

My next course of action was to see if any behavior triggered an alert in the firewall. Filtering by action, I found the following events:

10.131.31.51(unresolved)	10.131.16.5(sal-s-dc01.corp.svbank.com)	High_DB	Inf_Services	dns	udp	alert	**	high
10.150.160.74(unresolved)	10.139.0.10(sal.sip.corp.svbank.com)	i_VPN_CVO-DMVPN	i_VPN_In	sip	tcp	alert	**	low
10.150.160.74(unresolved)	10.139.0.10(sal.sip.corp.svbank.com)	i_VPN_CVO-DMVPN	i_VPN_In	sip	tcp	alert	**	low
172.31.1.236(unresolved)	74.125.28.95(pc-in-f95.1e100.net)	Guest_Wireless	Guest_Out	ssl	tcp	alert	"*.googleapis.com/"	informational
172.31.1.236(unresolved)	74.125.28.95(pc-in-f95.1e100.net)	Guest_Wireless	Guest_Out	ssl	tcp	alert	"*.googleapis.com/"	informational
10.138.16.17(unresolved)	10.138.32.5(SAL-UVS-DC03)	UAT_LB_App	UAT_Inf_Servcs	dns	udp	alert	**	high
10.138.16.57(unresolved)	10.138.32.5(SAL-UVS-DC03)	UAT_LB_App	UAT_Inf_Servcs	dns	udp	alert	**	high
10.122.22.24(obodb17ads.corp.svbank.com)	10.138.32.5(SAL-UVS-DC03)	Pre-Prod_In	UAT_Inf_Servcs	dns	udp	alert	**	high
10.131.31.10(sal-s-dc01.corp.svbank.com)	10.131.16.5(sal-s-dc01.corp.svbank.com)	High_DB	Inf_Services	dns	udp	alert	**	high

There are a few things that stand out. First, multiple DNS alerts with three separate source IPs sending traffic to a domain controller (SAL-UVS-DC03). Though not seen in the image due to space constraints, these were all sent within a very narrow timeframe. Because of the rapid timeframe, and the fact this behavior is not demonstrated anywhere else in the logs analyzed here, it is worth noting.

Second, two back-to-back VPN alerts from the same source IP. While two network sessions like this are normal and can indicate retry behavior, both network sessions were flagged as alerts. This is atypical of average network behavior.

Finally, a DNS event from a database to a domain controller via DNS. What makes this request atypical is the fact a DNS request from a database to a domain controller is being flagged as an alert. Combined with the sensitivity of a database, this is noteworthy.

At this point, I was able to name a few IPs that needed further investigation.

10.122.22.24/10.138.16.57/10.138.16.17 - Targeted a DC with DNS requests - short time frame.

10.131.31.51 - Database that is communicating via DNS to a DC.

139.131.195.10/208.80.234.202 - Both involved in the largest size event in the logs.

10.150.160.74 - Triggered back-to-back VPN alerts.

The additional piece of evidence I gathered was from the source IP communicating with the DC via DNS. The second event was this same IP (ending in .24) communicating with a Security Zone with syslog as the application. I was able to investigate the Security Zone IP and see that there were many other IPs tied to the same behavior (with 10.131.12.12 as the destination and using syslog). Evidence suggests this is tied to normal behavior, such as SIEM monitoring.

src_ip ↕	dest_ip ↕	src_zone ↕	dest_zone ↕	app ↕	protocol ↕	action ↕	bytes ↕	packets ↕
10.122.22.24(obodb17ads.corp.svbank.com)	10.131.12.12(unresolved)	Prod_In	Security	syslog	udp	allow	342	4
10.122.22.24(obodb17ads.corp.svbank.com)	10.138.32.5(SAL-UVS-DC03)	Pre-Prod_In	UAT_Inf_Servcs	dns	udp	alert	**	high

Further Investigation Recommendations

With the evidence collected, I would recommend gathering the pre-existing network logs/packets tracing surrounding the above-mentioned IPs, done either through an external tool like Wireshark or through internal tools like Microsoft Sentinel. The firewall logs, while useful, are not enough to tell the full story, and most enterprise networks monitor network and packet activity. If an attacker got into the system and performed data exfiltration, we need to fully understand how they broke in and how they moved laterally—information not captured in the firewall logs.

Based on the limited data I have, it is not possible to determine the methods of access and movement conclusively. The VPN could have been a point of access (VPN authentication logs required), as well as the unusual DNS activity. (supported through DNS query logs). Lateral movement is impossible to discuss without network and authentication logs, and these would be important to tell the story of the final large transfer out of the system.

Conclusion

No conclusive evidence of malicious behavior exists, but there are certainly outliers that merit a closer look. Multiple DNS requests were observed by different hosts to the DC that are tightly clustered together and inconsistent with the rest of the log. Multiple VPN requests from the same host back-to-back could be a simple retry attempt. A database sending DNS requests to a DC, while normal, warrants investigation due to its sensitive nature. These transactions are, however, more noteworthy as they were flagged as alerts in the firewall.